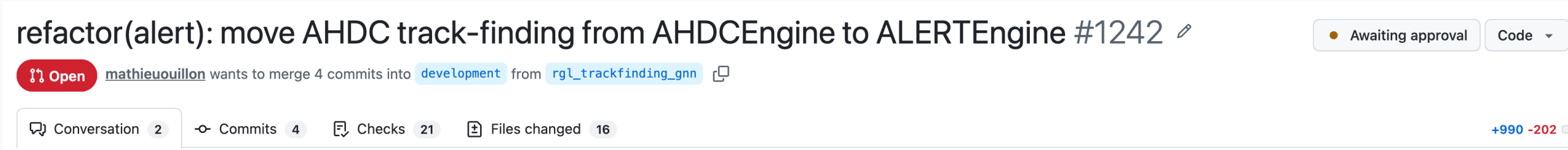
Graph Neural Network for ALERT

Mathieu Ouillon (Mississippi State University)

05/11/26

Pull request

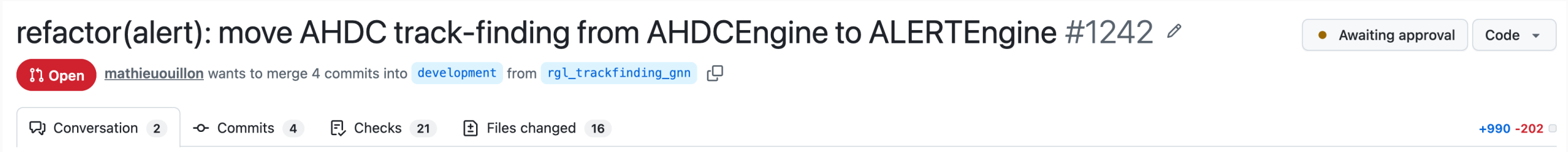
- Finish the pull request



Separation between engines:
AHDCEngine is slimmed down so it only reads AHDC::adc, applies calibration via HitReader, and writes AHDC::hits.
The entire track-finding pipeline, preclustering, the track finder itself, DOCA refinement, and helix fit, moves into ALERTEngine, where it now runs alongside the existing projection, matching, prePID, and Kalman stages on top of AHDC::hits + ATOF::hits/clusters.
The track-finder selection key in YAML changes from AHDC.Mode to ALERT.Mode, and the ATOF::tdc gate is moved so events without ATOF still produce AHDC::* banks

Pull request

- Finish the pull request

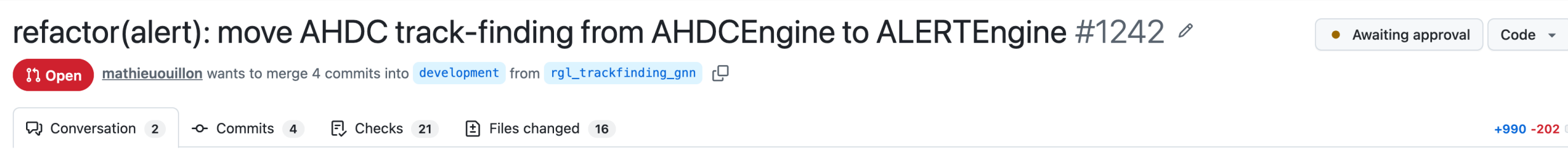


A strategy-pattern cleanup:
The three existing finders are unified behind a TrackFinder { findTracks(hits) -> TrackFinderResult } interface, with AITrackFinder, DistanceTrackFinder, and HoughTrackFinder each owning their preclustering and mode-specific logic. ALERTEngine becomes a thin dispatcher that picks the strategy in init() and calls findTracks(hits) once per event.

```
// II) Track Finding via the strategy selected in init() (ALERT.Mode YAML key).
// The implementation owns its own preclustering, cluster building, and any
// mode-specific safety fallbacks (e.g. AITrackFinder delegates to Distance
// when the hit count exceeds its MAX_HITS_FOR_AI threshold). The ATOF bank
// is passed for finders that build joint AHDC+ATOF graphs (GNN); the
// AHDC-only finders inherit the default and ignore it.
DataBank atofHitsBankForGNN = event.hasBank("ATOF::hits") ? event.getBank("ATOF::hits") : null;
TrackFinderResult trackResult = trackFinder.findTracks(AHDC_Hits, atofHitsBankForGNN);
if (!trackResult.isValid()) {
    return false;
}
ArrayList<Track> AHDC_Tracks = new ArrayList<>(trackResult.getTracks());
```

Pull request

- Finish the pull request



A new GNN-based track finder:
A fourth mode, GNN_Track_Finding, runs a GravNet edge scorer (TorchScript via DJL) over a per-event AHDC + ATOF hit graph, extracts tracks as connected components on edges with sigmoid score ≥ 0.1, then re-preclusters those hits into per-superlayer clusters so the existing DOCA refinement / helix fit / Kalman stages consume them unchanged.
The old AI_Track_Finding is renamed MLP_Track_Finding for clarity.

Pull request

- Finish the pull request

- Tested on 100 files (run 22991), with MLP or GNN track finding: no error

GNN vs MLP: Performance on elastic data

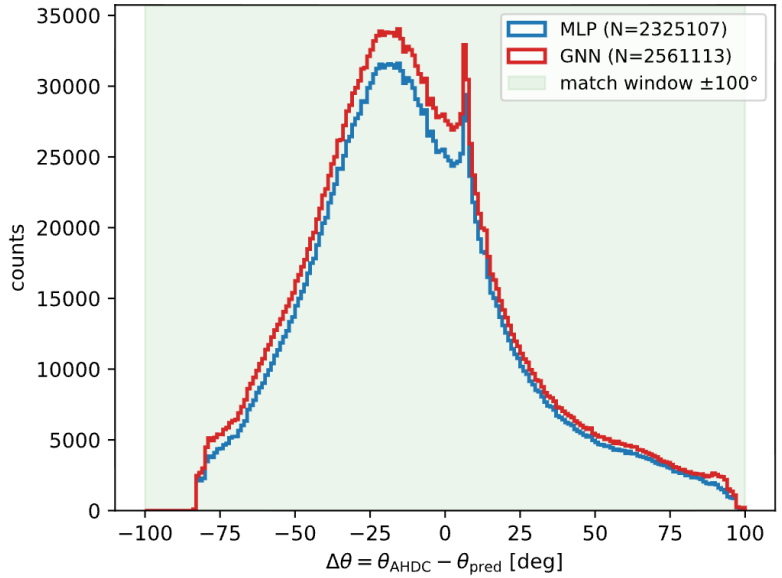
Elastic scattering on deuterium, use polar angle of the electron to compute it momentum and the expected kinematics of the recoil.

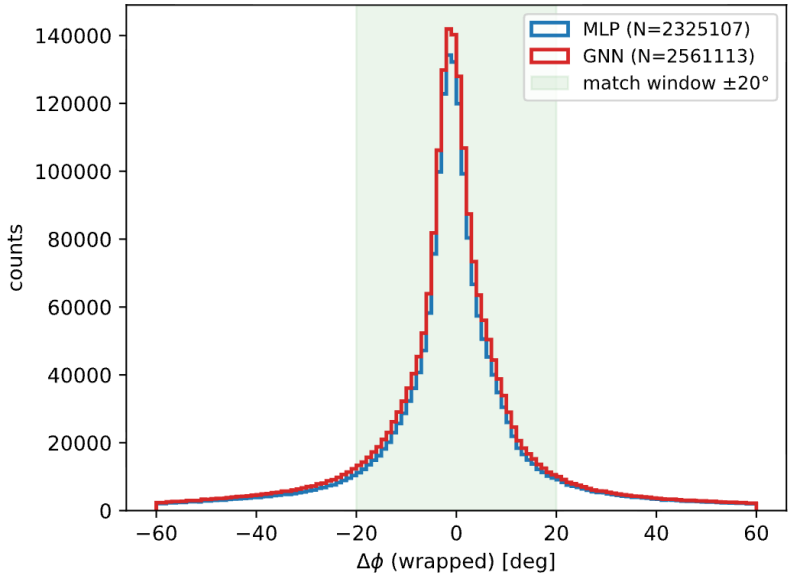
Look at the kinematics of the track found by MLP and GNN:

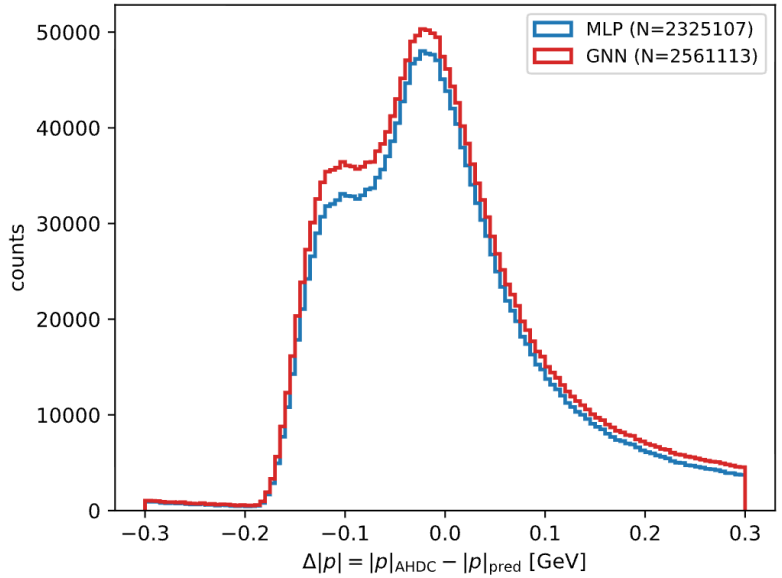
- A track is match when both angular residuals are inside the matching windows

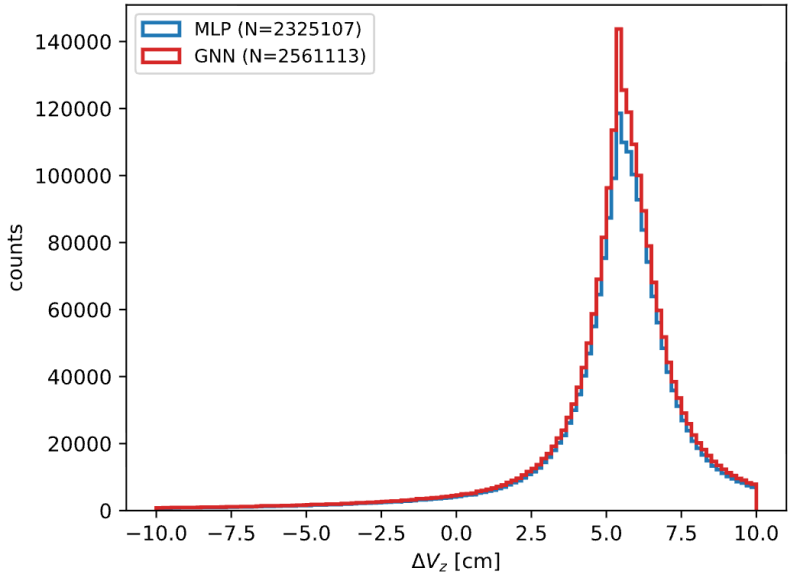
- GNN found more tracks that pass the Helix fitter than MLP

- With no specific phase space



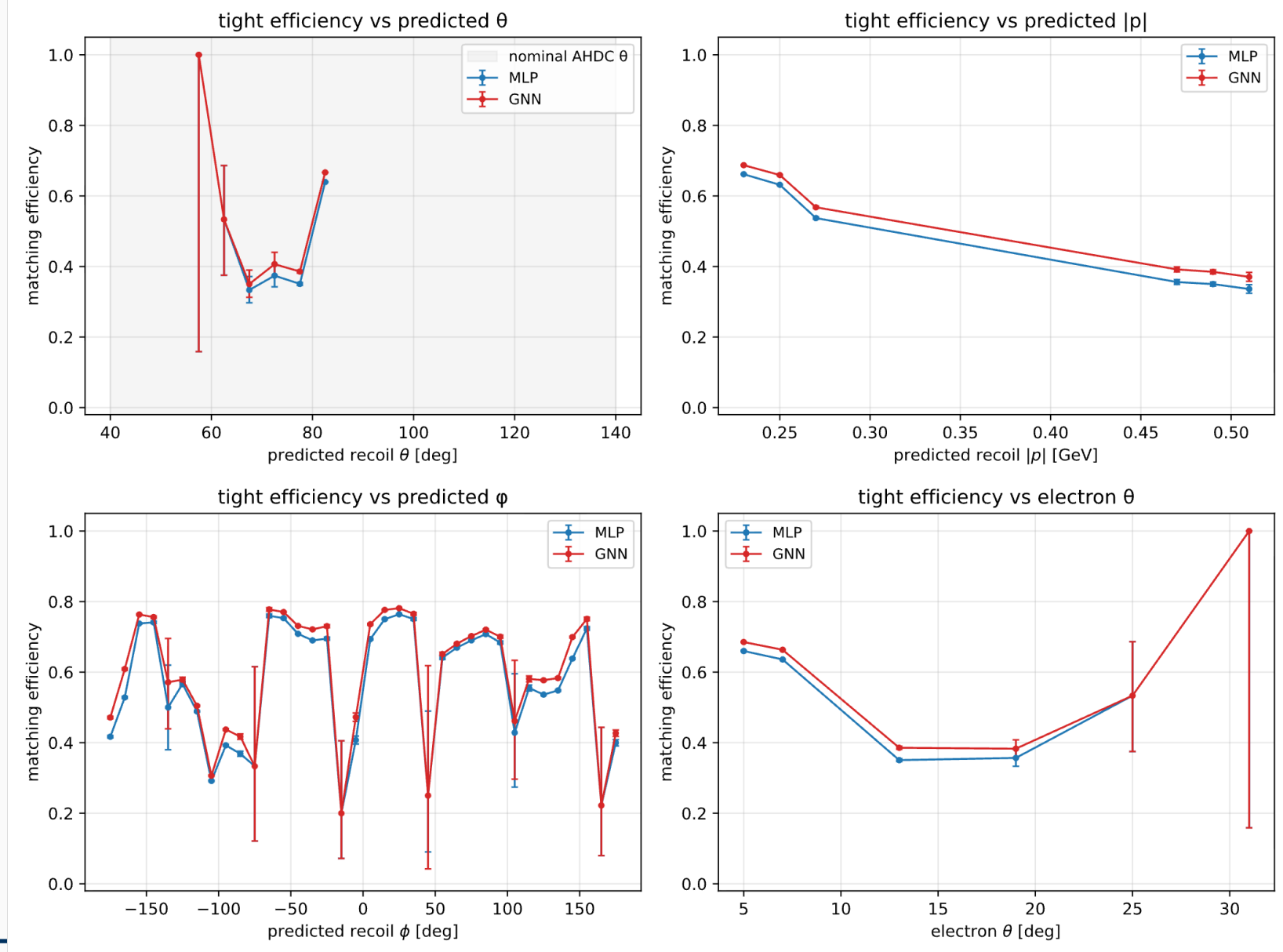






GNN vs MLP: Performance on elastic data

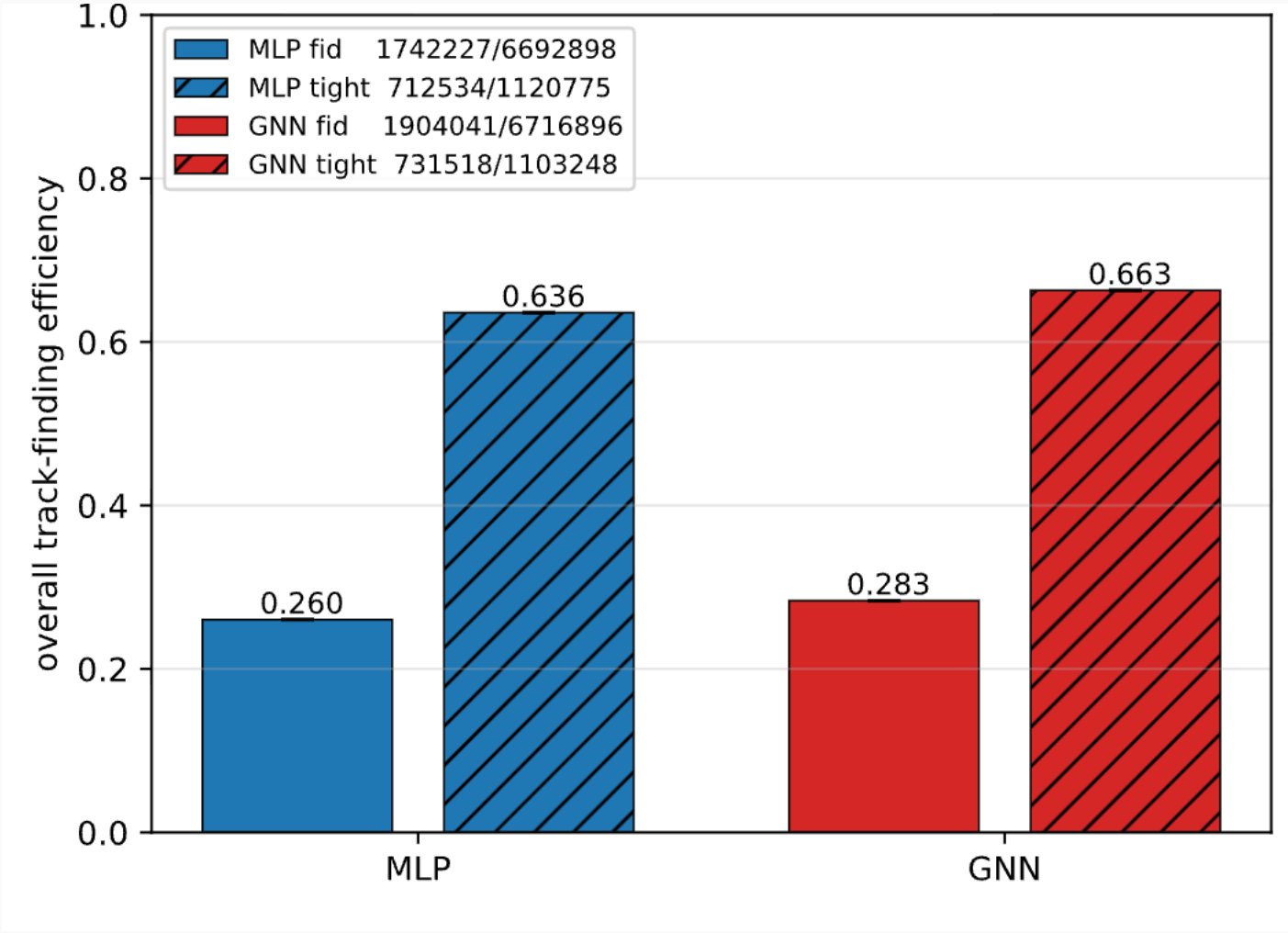
Can compute the efficiency:
- Tight track-finding efficiency (denom: elastic & in fiducial & 7 hits in 7 layers within ±30° of predicted)



GNN vs MLP: Performance on elastic data

Can compute the efficiency:

- Tight track-finding efficiency (denom: elastic & in fiducial & 7 hits in 7 layers within ±30° of predicted)



Summary

- GNN find more tracks
- GNN is a bit slower, but no significantly (0.16% of a standard reconstruction time)
- GNN have advantages:
  - Work at the hit level, so no clustering
  - Find tracks compose of AHDC and ATOF hits
  - Find more tracks (the difference in efficiency is probably higher in event with higher multiplicity, see simulation)
- Pull request to implement that